

Light-it QA Challenge

1. Test Strategy

Objective

The objective of this test is to validate that a patient can successfully reschedule an existing appointment to another available time slot without cancelling and creating a new appointment.

The testing will focus not only on the successful rescheduling flow, but also on appointment availability, data integrity, appointment states, error handling, and the consistency between the UI and backend data.

Scope

- Existing appointments
- Appointment statuses
- Available and unavailable time slots
- Rescheduling flow
- Date and time selection
- Persistence of the rescheduled appointment
- Appointment list and details
- Validation and error handling
- Boundary and negative scenarios
- Impact on the original appointment
- Prevention of duplicate appointments or conflicting bookings
- UI/backend consistency

Out of scope

- Performance testing
 - Security testing
 - Cross-browser compatibility
-

2. Test Approach

A. Happy Path

The primary user journey was validating the following Scenario:

Scenario: A patient successfully reschedules an appointment
Given A patient is logged on the Medical Appointment platform
And The patient is on the appointments page
And The patient has at least one appointment
When The patient clicks on the Reschedule button
And The patient selects a date and a time
And The patient clicks on Confirm button
Then The UI shows a success message
And The appointment shows the new date and time selected

The expected outcome is that the existing appointment's date and time are modified while preserving the relevant appointment and patient information.

B. Appointment Status

The following appointment statuses were identified during exploration:

- Completed
- Cancelled
- Active
- Confirmed

All observed appointment statuses displayed a **Reschedule** action in the UI.

Since the application exposes the rescheduling action for all of these states, each state was considered relevant to the rescheduling flow. However, the business rule regarding which appointment states should actually be reschedulable could not be independently confirmed.

This should therefore be considered a business-rule assumption rather than a confirmed requirement.

C. Availability

The following scenarios were considered relevant:

- Rescheduling to an available slot
- Rescheduling to an unavailable slot
- Attempting to reschedule when a previously available slot becomes unavailable
- Rescheduling on the same date
- Rescheduling to a different date
- Attempting to select a past date/time
- Attempting to select the current time
- Rescheduling when the selected day is fully booked
- Attempting to reschedule to a slot already occupied by another appointment.

Particular attention was given to appointment availability because allowing two appointments to occupy the same slot could result in a significant business and operational impact.

D. Data Integrity

The rescheduling operation was also evaluated from a data integrity perspective.

Before rescheduling

Appointment ID: 1
Date: Aug 28
Time: 10:00
Status: Active

After rescheduling

Appointment ID: 1
Date: Aug 30
Time: 15:00
Status: Active

The following conditions should be satisfied:

- The appointment ID remains unchanged.
- No additional appointment is created.
- The appointment no longer occupies the original time slot.
- The new time slot becomes occupied by the appointment.
- Patient information remains unchanged.
- The doctor remains unchanged.
- The appointment status remains unchanged.
- The UI reflects the same appointment state and information as the backend.

4. Bugs

BUG-001 — Rescheduled appointment displays old information in the UI

Severity: High

Priority: High

Environment

Production environment / Chrome

Preconditions

- The patient is logged in.
- The patient has at least one existing appointment.

Steps to reproduce

1. Click the **Reschedule** button for an existing appointment.
2. Select a different available date and time.
3. Click **Confirm**.
4. Observe the appointment information displayed in the UI.
5. Refresh the page.
6. Observe the appointment information again.

Expected Result

The UI should immediately display the newly selected appointment date and time after a successful rescheduling operation. The information should remain consistent after refreshing the page.

Actual Result

The UI does not automatically update the appointment information after a successful rescheduling operation.

After refreshing the page, the appointment information is updated, but the persisted date may differ from the date selected during the rescheduling process. This behavior is further described in BUG-002.

Impact

This creates a discrepancy between the state displayed to the patient and the actual persisted appointment. A patient may believe that the appointment was not successfully rescheduled or may rely on incorrect appointment information.

Evidence

See Video 1

BUG-002 — Selected appointment date is persisted as the previous calendar day on the UI

Severity: High

Priority: High

Environment

Production environment / Chrome

Preconditions

- The patient is logged in.
- The patient has at least one existing appointment.

Steps to reproduce

1. Click the **Reschedule** button for an existing appointment.
2. Select a future date from the calendar. For example, select **August 29**.
3. Select an available time.
4. Click **Confirm**.
5. Refresh the page.
6. Observe the appointment date.
7. Check the same date on the DB, appointments table.

Expected Result

The appointment should be displayed on the exact date selected by the patient.

For example:

Selected:
August 29
15:00

The persisted appointment should remain:

UI: August 29
DB: August 29
15:00

Actual Result

The selected time is preserved, but the appointment date is persisted and displayed as the **previous calendar day**. Note: It has been discovered that the value on the database is the selected one.

For example:

Selected:
August 29
15:00

After refresh:
UI: August 28
DB: August 29
15:00

This behavior is consistently reproducible when selecting a different calendar date.

Impact

This is a critical issue because the patient may attend the appointment on the wrong day. The problem directly affects the core business functionality of appointment scheduling and can potentially result in missed appointments and operational conflicts.

Possible relationship to BUG-003

This behavior may be related to the timestamp/timezone issue described in BUG-003. However, the root cause has not been confirmed.

Possible areas to investigate include:

- Client-side timezone conversion
- Server-side timezone handling
- Date serialization/deserialization
- UTC/local-time conversion

Evidence

See Video 1

BUG-003 — Appointment updated_at timestamp is inconsistent with the actual updated time

Severity: Medium

Priority: Medium

Environment

Production environment / Chrome

Preconditions

- The patient is logged in.
- The patient has at least one existing appointment.

Steps to reproduce

1. Click the **Reschedule** button for an existing appointment.
2. Select a different date and time.
3. Click **Confirm**.
4. Inspect the appointment's `updated_at` field.
5. Refresh the website.
6. Inspect the `updated_at` field returned by the appointments endpoint.

Expected Result

The `updated_at` timestamp should accurately represent when the appointment was updated, using the application's documented timezone convention.

Actual Result

The `updated_at` field contains a timestamp corresponding to the previous day and approximately 12 hours earlier than the expected update time.

Impact

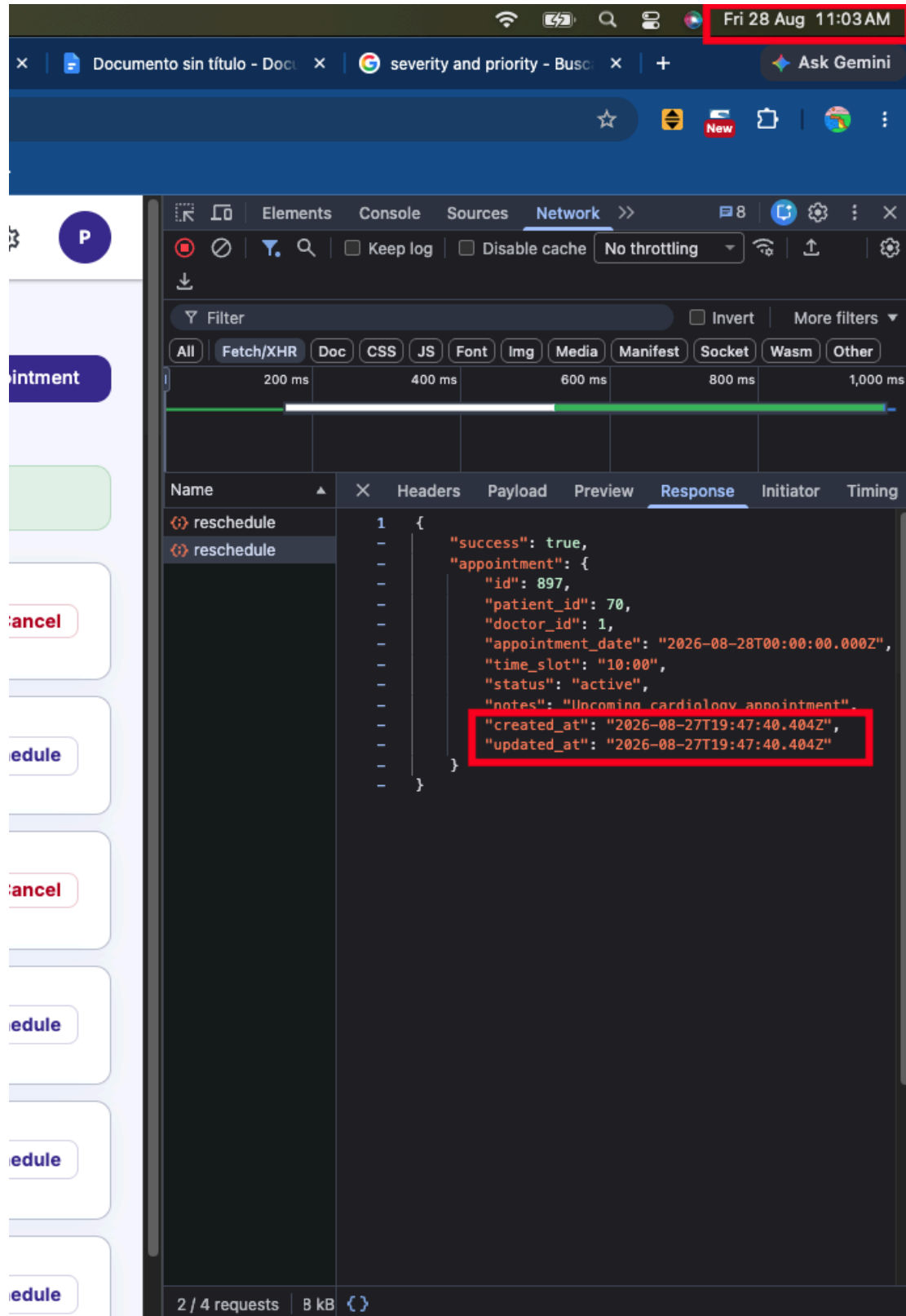
An incorrect or inconsistently interpreted timestamp can affect auditability, troubleshooting, integrations, notifications, and any functionality that relies on appointment modification timestamps.

Note

The observed behavior may be related to timezone handling rather than the timestamp itself being technically incorrect.

Because BUG-002 shows that the **appointment date itself shifts to the previous day**, both issues may have a common underlying cause. However, further investigation is required before establishing timezone handling as the root cause.

Evidence



BUG-004 — Confirm button allows multiple rescheduling requests

Severity: Medium

Priority: Medium

Environment

Production environment / Chrome

Preconditions

- The patient is logged in.
- The patient has at least one existing appointment.

Steps to reproduce

1. Open an existing appointment.
2. Click **Reschedule**.
3. Select an available date and time.
4. Double-click the **Confirm** button.
5. Open the browser Network tab and inspect the requests.

Expected Result

Only one rescheduling operation should be submitted for a single user action.

The Confirm button should be disabled or otherwise protected against duplicate submissions while the request is being processed.

Actual Result

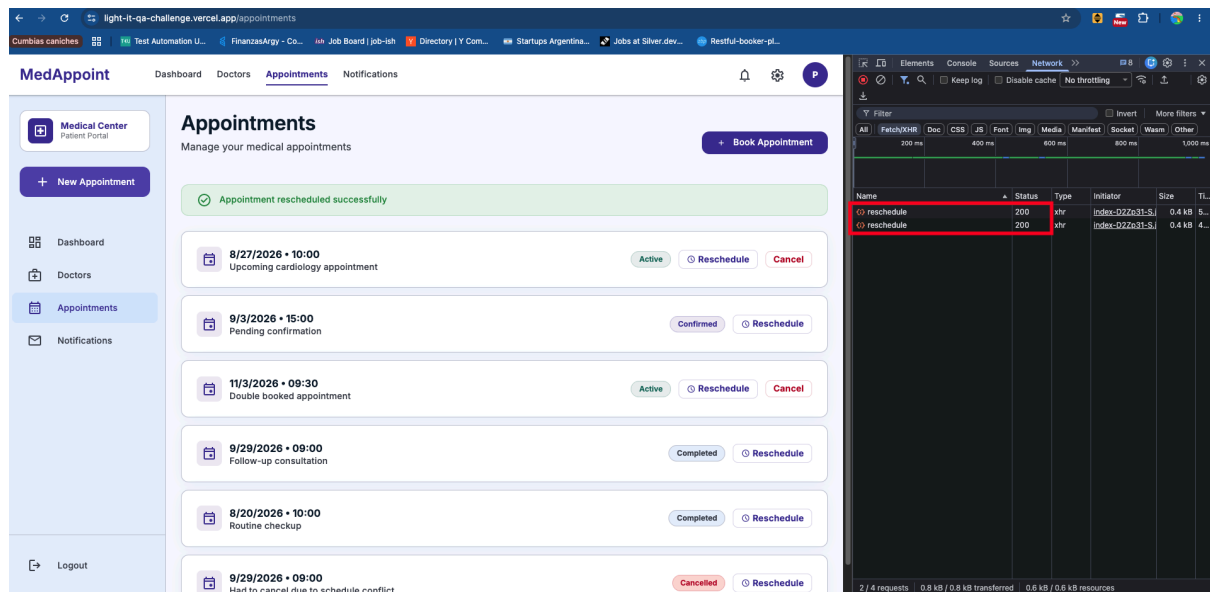
Two rescheduling API requests are sent when the Confirm button is double-clicked.

Impact

Duplicate requests can result in unintended side effects, such as duplicate notifications, duplicated downstream events, inconsistent asynchronous processing, or other unexpected behavior depending on how the backend handles repeated requests.

The backend should also ideally protect the operation through idempotency or appropriate concurrency controls rather than relying exclusively on the UI.

Evidence



BUG-005 — System allows conflicting appointments for the same time slot

Severity: High

Priority: High

Environment

Production environment / Chrome

Preconditions

- The patient is logged in.
- The patient has at least two appointments.
- One appointment occupies the target date/time slot.

Steps to reproduce

1. Open an existing appointment.
2. Click **Reschedule**.
3. Select a date and time that is already occupied by another appointment.
4. Click **Confirm**.
5. Refresh the page and verify the resulting appointments.

Expected Result

The system should prevent the conflicting reschedule operation.

At the API level, the request should return an appropriate conflict response, such as HTTP **409 Conflict**, if that is the API's defined contract.

The UI should display an appropriate message, such as:

"Sorry. This time slot is already booked."

Actual Result

The UI displays:

"Appointment rescheduled successfully"

and allows the appointment to be moved to a slot that is already occupied.

Impact

This can result in two appointments being assigned to the same time slot. Depending on the business rules, this could lead to double booking, scheduling conflicts, operational issues, and a poor patient experience.

Evidence

The screenshot displays the MedAppoint web application interface. The main content area shows a list of appointments under the heading "Appointments". The appointments are as follows:

Date & Time	Description	Status	Actions
8/29/2026 • 09:30	Upcoming cardiology appointment	Active	Reschedule, Cancel
9/3/2026 • 15:00	Pending confirmation	Confirmed	Reschedule
11/3/2026 • 09:30	Double booked appointment	Active	Reschedule, Cancel
9/29/2026 • 09:00	Follow-up consultation	Completed	Reschedule
8/20/2026 • 10:00	Routine checkup	Completed	Reschedule
9/29/2026 • 09:00	Had to cancel due to schedule conflict	Cancelled	Reschedule

Two appointments are highlighted with red boxes: "9/29/2026 • 09:00 Follow-up consultation" and "9/29/2026 • 09:00 Had to cancel due to schedule conflict".

On the right side of the screenshot, a network console is visible, showing a list of requests. The first request is named "appointments" with a status of 200, type of xhr, and a size of 0.5 kB.

BUG-006 — Reschedule endpoint returns old appointment data in the response

Severity: High

Priority: High

Environment

Production environment / API

Preconditions

- The patient is logged in.
- The patient has an existing appointment.
- A different available date and time can be selected for rescheduling.
- Open network tab.

Steps to reproduce

1. Select an existing appointment.
2. Click **Reschedule**.
3. Select a different date and time.
4. Click **Confirm**.
5. Inspect the response of the **PUT** reschedule endpoint.

Expected Result

The **PUT** reschedule endpoint should return the updated appointment object, including the newly selected date and time.

For example, if the appointment was rescheduled to **November 5 at 10:30**, the response should contain:

```
{
  "success": true,
  "appointment": {
    "id": 899,
    "appointment_date": "2026-11-05T00:00:00.000Z",
    "time_slot": "10:30"
  }
}
```

The response should also contain an updated **updated_at** value reflecting the rescheduling operation.

Actual Result

The endpoint returns `success: true`, but the `appointment` object contains the **previous appointment date, time, and `updated_at` timestamp** instead of the newly submitted values.

Example response:

```
{
  "success": true,
  "appointment": {
    "id": 899,
    "patient_id": 70,
    "doctor_id": 1,
    "appointment_date": "2026-11-04T00:00:00.000Z",
    "time_slot": "09:30",
    "status": "active",
    "notes": "Double booked appointment",
    "created_at": "2026-08-27T19:47:44.753Z",
    "updated_at": "2026-08-27T19:47:44.753Z"
  }
}
```

Impact

The API reports a successful operation while returning stale appointment data. Consumers of the endpoint may therefore assume that the rescheduling operation did not occur or may overwrite the newly updated state with outdated information.

This can also contribute to inconsistencies between the API response, frontend state, and persisted appointment data.

Additional Observation

This behavior may be related to **BUG-001 (old UI information)** and the date/timestamp issues described in **BUG-002 and BUG-003**. However, the common root cause has not been confirmed.

Further investigation should verify:

- The request payload sent by the client.
- The database state immediately after the update.
- The object returned by the backend after the update.
- Whether the endpoint returns a stale object before the database transaction/update has been fully reflected.
- Whether caching or ORM/entity state is involved.

Evidence

CompletedReschedule

Reschedule

Reschedule

Cancel

CancelledReschedule

Reschedule

CompletedReschedule

Reschedule

Name	×	Headers	Payload	Preview	Response	Initiator	Timing
availability			Request Payload		View source		
reschedule							
availability							
reschedule							

```
{
  "appointment_date": "2026-08-06",
  "time_slot": "14:00"
}
```

Reschedule

Reschedule

Reschedule

Cancel

Reschedule

Reschedule

Reschedule

Reschedule

Name	×	Headers	Payload	Preview	Response	Initiator	Timing
availability	1						
reschedule	-						
availability	-						
reschedule	-						

```
{
  "success": true,
  "appointment": {
    "id": 897,
    "patient_id": 70,
    "doctor_id": 1,
    "appointment_date": "2026-09-15T00:00:00.000Z",
    "time_slot": "10:30",
    "status": "active",
    "notes": "Upcoming cardiology appointment",
    "created_at": "2026-08-27T19:47:40.404Z",
    "updated_at": "2026-08-27T19:47:40.404Z"
  }
}
```

5. Improvements

IMPROVEMENT-001 — Provide explicit feedback while the rescheduling request is being processed

The Confirm button remains interactive after it is pressed.

Recommendation

Disable the Confirm button immediately after submission and provide a loading state until the operation completes.

This would improve the user experience and reduce accidental duplicate submissions.

Note: This UI improvement should complement, not replace, backend protection against duplicate requests.

IMPROVEMENT-002 — Provide clearer confirmation of the updated appointment

After a successful rescheduling operation, the user should receive clear confirmation containing the new appointment date and time.

For example:

"Your appointment has been rescheduled to August 30 at 15:00."

This would make the result of the operation immediately clear and reduce uncertainty for the patient.

IMPROVEMENT-003 — Improve data types for statuses

On the database level, the status field for appointments and payments are defined as strings. This definition is not the best for this type of system. It would be much better to define them as enums

IMPROVEMENT-004 — Mixed date formats in the platform

Within the same page, we can see the appointment's date as MM-DD-YYYY (american) while in the reschedule panel, right below, the format is DD-MM-YYYY (not american).

The date format should be the same across all the UI.

IMPROVEMENT-005 — Reschedules to past days shouldn't be allowed

A patient can reschedule an appointment to a past date. This shouldn't be allowed as it is not possible to attend.

6. Quality Assessment

The primary rescheduling flow works successfully for the basic happy-path scenario. However, several issues were identified that affect the reliability, data consistency, and usability of the feature.

The most significant issue is that the system allows an appointment to be rescheduled to a time slot that is already occupied by another appointment. This represents a direct risk to appointment availability and could result in double booking.

A second critical issue is that the date selected by the patient can be persisted as the previous calendar day. For example, selecting August 29 results in the appointment being displayed as August 28 after refreshing the page, while the selected time is preserved. This directly affects the core purpose of the feature because the patient may attend the appointment on the wrong day.

Additionally, the UI temporarily displays old appointment information immediately after a successful rescheduling operation, creating a discrepancy between the information shown to the patient and the persisted backend state.

An inconsistency was also identified with the `updated_at` timestamp. This may be related to the same timezone/date conversion issue affecting the appointment date, but the root cause has not been confirmed.

Finally, the UI allows multiple rescheduling requests to be generated through a double-click on the Confirm button. Although the UI should prevent accidental duplicate submissions, the backend should also provide appropriate protection against duplicate operations.

Risk Assessment

Area	Risk	What's happening
Core Rescheduling Flow	High	The basic flow works, but selected dates can be persisted incorrectly
Appointment Availability	Critical	Conflicting appointments can be created
Data Integrity	Critical	Appointment dates can differ from the user's selection
Error Handling	High	Conflicting slot selection is not correctly prevented
User Experience	High	Patients can be shown incorrect or old appointment information

7. Go / No-Go Recommendation

Recommendation: **NO-GO**

Based on the scenarios executed, the rescheduling feature should **not be released in its current state**.

There are multiple issues affecting the reliability of the core functionality.

The primary release blockers are:

1. **The UI does not show the correct date and time after a reschedule.**
2. **The selected appointment date can be persisted as the previous calendar day.**
3. **The system allows conflicting appointments to be created for the same time slot.**

These behaviors can directly affect patient's appointments and can result in patients receiving or relying on incorrect appointment information.

Additionally, the UI can display stale appointment information immediately after a successful rescheduling operation, and double-clicking the Confirm button generates multiple rescheduling requests.

The `updated_at` timestamp issue should also be investigated, particularly in conjunction with the appointment date discrepancy, to determine whether both issues originate from a common timezone or date serialization problem.

The feature should be reconsidered for release once the appointment conflict behavior and incorrect date persistence are fixed, and the identified data consistency and

duplicate-submission risks have been addressed or formally accepted by the product/business stakeholders.